

CLOUDSHIELD AI

System Architecture Specification

Document 02 | Product-Level Technical Architecture | Version 1.0

Field	Value
Document ID	CSA-02-SAS
System	CloudShield AI
Type	System Architecture Specification
Version	1.0
Status	Architecture Baseline / Pre-Implementation
Parent	CSA-00-MSS + CSA-01-SRS
Scope	Complete CloudShield AI platform
DAIL Position	Dedicated subsystem within the application/platform architecture

Architectural principle: separate product concerns, isolate trust boundaries, make state explicit, and keep AI non-authoritative.

Document Control & Contents

#	Section
1	1. Architecture Purpose & Principles
2	2. Architectural Drivers
3	3. System Context
4	4. Logical Architecture
5	5. Container / Service Architecture
6	6. Frontend Architecture
7	7. API & Application Architecture
8	8. Cloud Discovery & Asset Architecture
9	9. Security & Risk Architecture
10	10. Dependency & Graph Architecture
11	11. AI Intelligence Architecture
12	12. Remediation Architecture
13	13. DAIL Subsystem Architecture
14	14. Verification & Promotion Architecture
15	15. Data Architecture
16	16. Eventing & Asynchronous Processing
17	17. Cloud Integration Architecture
18	18. Security Architecture & Trust Boundaries
19	19. Identity, Authorization & Tenant Isolation
20	20. Evidence, Audit & Observability Architecture
21	21. Infrastructure & Deployment Architecture
22	22. Reliability & Failure Architecture
23	23. Scalability & Performance Architecture
24	24. Configuration & Policy Architecture
25	25. Environment Architecture
26	26. Technology Selection Principles
27	27. API / Service Contracts
28	28. Architectural Data Flows
29	29. Deployment Flow
30	30. Architecture Decisions & Tradeoffs
31	31. MVP Architecture
32	32. Future Evolution
33	33. Architecture-to-Requirements Traceability
34	34. Architecture Acceptance Criteria

1. Architecture Purpose & Principles

This specification defines the technical architecture of CloudShield AI as a complete product. It establishes logical layers, service boundaries, data ownership, integration boundaries, trust boundaries, deployment topology, and the relationship between AI, deterministic application logic, cloud integrations, and DAIL.

1.1 Core Architectural Principles

- Product-first architecture: CloudShield AI is the system; DAIL is a subsystem.
- Explicit state: observed, proposed, verified, and trusted state must remain distinguishable.
- AI is non-authoritative: model output becomes actionable only through structured validation and lifecycle controls.
- Deterministic safety: security rules, invariants, policy gates, verification, and promotion must not depend solely on model judgment.
- Least privilege: each user, service, integration, and tool receives only required permissions.
- Traceability: important state transitions and AI-assisted actions must be correlatable to evidence.
- Modularity: cloud-provider, LLM-provider, and DAIL internals must be replaceable behind explicit contracts.
- Asynchronous by default for long-running cloud discovery, AI jobs, verification, and promotion workflows.
- Fail closed on safety gates: inability to prove required conditions must not produce trusted state.
- Implementation simplicity for MVP: use a small number of well-defined services/modules before introducing distributed complexity.

2. Architectural Drivers

Driver	Architectural consequence
Cloud security	Strong IAM, isolation, audit, encrypted sensitive data, bounded cloud operations.
AI assistance	AI abstraction, context control, structured outputs, provider isolation.
Deterministic lifecycle	DAIL boundary, explicit candidates, verification and promotion states.
Asset relationships	Normalized asset model plus dependency graph.
Auditability	Durable evidence and append-oriented audit trail.
Long-running workflows	Queues/jobs/workers and explicit status.
MVP delivery	Modular monolith or small service set preferred over premature microservices.
Future scale	Stateless API/services, separable workers, partitionable data and event-driven extensions.

3. System Context

CloudShield AI sits between operators/automation and external cloud/control-plane systems. The platform reads cloud state, creates normalized internal state, analyzes security posture, uses AI for contextual reasoning, creates remediation candidates, and controls infrastructure-affecting actions through deterministic lifecycle gates.

High-level context:

Users / Automation → CloudShield AI UI/API → Application Core → Intelligence + Security + DAIL → Cloud Provider APIs / External Integrations

External system	Interaction
AWS / cloud provider	Discover resources, read configuration, validate state, execute authorized changes.
LLM provider	Generate explanations, recommendations, and structured remediation proposals.
Identity provider	Authenticate users and optionally provide enterprise identity.
Source control	Store or reference infrastructure/configuration artifacts.
CI/CD	Trigger verification or consume lifecycle outcomes.
Ticketing/notification	Receive findings, decisions, or workflow events.

4. Logical Architecture

The canonical logical architecture is:

Experience Layer



API & Application Layer



CloudShield Product Core



Security / Risk / Dependency / Remediation Engines



AI Intelligence + DAIL



Data / Evidence / Eventing



Cloud & External Integrations

Layer	Primary responsibility	Authority
Experience	User interaction, visualization, review.	No direct infrastructure authority.
API/Application	Auth, orchestration, workflows, policy enforcement.	Authoritative workflow coordinator.
Product Core	Organizations, projects, environments, assets, findings, candidates.	Authoritative product state.
Security/Risk	Rules, findings, deterministic risk signals.	Authoritative for generated rule results.
Dependency	Relationships and impact context.	Authoritative for modeled dependency state.
AI	Contextual reasoning, explanations, recommendations, candidate generation.	Non-authoritative.
DAIL	Candidate lifecycle, invariants, impact, verification, promotion.	Authoritative for lifecycle gates.
Data/Evidence	Persist state and proof.	System of record for stored artifacts.
Integrations	Cloud/model/external communication.	External authority remains external.

5. Container / Service Architecture

For the MVP, the architecture SHOULD begin as a modular application with separately deployable workers where workload characteristics justify it. The logical boundaries are:

Component	Responsibilities	State owned
Web Application	Dashboard, asset explorer, findings, remediation, lifecycle UI.	UI/session state only.
API Gateway / API Layer	Routing, auth, validation, rate limiting, API contracts.	Request/job metadata as needed.
Application Core	Organizations, projects, environments, workflow orchestration.	Product entities.
Discovery Worker	Cloud inventory collection and normalization.	Discovery job state.
Security Engine	Rule evaluation and finding generation.	Finding/rule results.
Risk Engine	Deterministic risk calculation and prioritization.	Risk assessment.
Dependency Engine	Relationship construction and traversal.	Dependency graph state.
AI Service	Context assembly, model invocation, structured response handling.	AI invocation metadata.
Remediation Engine	Candidate construction and revision management.	Candidate state.
DAIL Engine	Lifecycle, invariants, impact, verification, promotion.	DAIL lifecycle state.
Evidence Service	Evidence storage/indexing and references.	Evidence metadata.
Event/Job System	Queues, retries, scheduling, asynchronous workflow state.	Job/event state.
Data Layer	Transactional product persistence and object storage.	Persistent system state.
Cloud Adapter	Provider-specific APIs and execution	Integration metadata.

	controls.	
--	-----------	--

6. Frontend Architecture

- Frontend SHALL communicate through authenticated APIs rather than directly accessing cloud provider credentials.
- UI state should distinguish observed state, proposed candidate state, verification result, and trusted state.
- Long-running jobs should use polling, server-sent events, or equivalent status/event mechanisms rather than blocking page requests.
- Role-based UI controls should hide or disable unauthorized actions, while server-side authorization remains authoritative.
- AI explanations should show provenance/context references where appropriate.
- Candidate review should expose impact, verification, policy requirements, and evidence before promotion.

6.1 Primary UI Domains

UI domain	Core views
Overview	Posture, findings, remediation, lifecycle, health.
Cloud	Accounts, environments, regions, asset inventory.
Security	Findings, severity, risk, evidence.
Assets	Configuration, dependencies, history.
AI	Explanation, recommendations, candidate generation.
Remediation	Candidates, versions, impact, verification.
DAIL	Lifecycle state, gates, promotion history.
Evidence	Artifacts, verification proof, audit events.
Experiments	Trials, model/prompt versions, metrics.
Administration	Users, roles, policies, integrations.

7. API & Application Architecture

The API layer is the controlled entry point for product actions.

API boundary	Examples
Identity	/auth, /users, /roles
Organizations	/organizations, /projects, /environments
Cloud	/cloud-accounts, /connections, /discovery-jobs
Assets	/assets, /configurations, /dependencies
Security	/findings, /rules, /risk
AI	/ai/explanations, /ai/recommendations
Remediation	/candidates, /candidate-versions
DAIL	/lifecycle, /impact, /verification, /promotion
Evidence	/evidence, /audit-events
Experiments	/experiments, /trials, /evaluations
Operations	/jobs, /health, /metrics/status

Mutating APIs must enforce authorization, schema validation, idempotency where appropriate, auditability, and safe error semantics.

8. Cloud Discovery & Asset Architecture

Cloud provider-specific discovery must be isolated behind adapters.

Flow: Cloud Adapter →Raw Provider Objects →Normalizer →Identity Resolver →Asset Store →Configuration Snapshot → Security/Dependency Consumers.

- Provider identifiers are retained for reconciliation.
- Normalized assets use product-level identities.
- Discovery is incremental where possible and full reconciliation is available for recovery.
- Partial failures are represented explicitly.
- Configuration snapshots are immutable/versioned observations.

- Provider-specific fields may be retained as extensions without contaminating the core domain model.

9. Security & Risk Architecture

Security analysis is a deterministic product capability. AI can explain or prioritize context but does not replace rule evaluation.

Subsystem	Input	Output
Rule Engine	Configuration/assets/policies	Finding candidates
Finding Manager	Rule results/evidence	Finding lifecycle
Risk Engine	Finding + asset context + exposure + criticality + dependencies	Risk assessment
Evidence Collector	Observed state/rule result	Detection evidence
AI Context Engine	Authorized finding/risk/evidence context	AI-ready context

10. Dependency & Graph Architecture

Dependency intelligence provides the relationship layer required for blast-radius reasoning and DAIL impact analysis.

- Relationships must have a defined type.
- Relationship provenance should identify whether the edge came from cloud observation, configuration, policy, or derived inference.
- Graph traversal must be bounded to prevent uncontrolled expansion.
- Impact analysis must preserve the distinction between directly modified assets and transitively affected assets.
- Dependency state should be versionable or time-aware when historical reconstruction is required.

11. AI Intelligence Architecture

The AI architecture is deliberately provider-independent:

AI Interface → Context Builder → Policy/Context Filter → Model Adapter → Structured Output Parser → Validation → Product Consumer

AI component	Responsibility
AI Interface	Stable internal contract used by product features.
Context Builder	Assemble only relevant authorized context.
Policy/Context Filter	Remove prohibited/sensitive context and enforce task limits.
Model Adapter	Translate internal requests to provider-specific API format.
Model Runtime	Perform inference.
Output Parser	Convert model output into typed structures.
Validator	Schema, policy, safety, and semantic validation.
Consumer	Explanation, recommendation, or remediation workflow.

The architecture must permit a hosted model today and a local/open-weight model later without changing the surrounding product contracts.

12. Remediation Architecture

Remediation is split into generation, normalization, validation, lifecycle, and execution.

Finding → Remediation Strategy → AI/Human Proposal → Candidate Normalization → Schema Validation → DAIL → Verification → Authorized Execution/Promotion.

- Candidates must be versioned.
- Candidate content must be machine-readable where execution is possible.
- Human-readable explanation and machine-executable representation should be stored separately.
- Execution authority must come from policy-controlled services, not model text.

- Every candidate must retain its origin and motivating evidence.

13. DAIL Subsystem Architecture

DAIL is a bounded subsystem. Its product-level interface is:

Candidate + Context → Identity → Dependency/Impact → Invariants → Verification → Policy Decision → Promotion → Trusted State + Evidence

DAIL module	Responsibility
Candidate Intake	Accept and validate structured candidates.
Identity	Assign/verify deterministic candidate/resource identity.
Dependency/Impact	Determine affected resources and lifecycle impact.
Invariant Engine	Evaluate required state conditions.
Verification Engine	Run deterministic validation gates.
Promotion Controller	Apply policy-approved state transition.
Evidence Recorder	Persist lifecycle proof.
State Store	Maintain DAIL lifecycle state.

The internal DAIL design is specified in the existing DAIL engineering documents and is not duplicated here.

14. Verification & Promotion Architecture

Verification is an explicit gate between proposed and trusted state.

Candidate → Static Validation → Plan/Preview Checks → Dependency/Impact Checks → Security Checks → Functional/Infrastructure Tests → Policy Evaluation → Approval → Promotion

- Verification steps are environment and change-class aware.
- Mandatory gate failure blocks promotion.
- Verification results are immutable records or protected revisions.
- Promotion creates a new trusted-state reference.
- Rollback or recovery is part of the architecture for supported high-risk changes.

15. Data Architecture

A transactional relational store is the default MVP system-of-record pattern for structured product state. Object storage is used for larger evidence/artifacts.

Data store concern	Recommended responsibility
Transactional database	Users, organizations, projects, environments, accounts, assets, findings, risks, candidates, lifecycle metadata.
Object storage	Large evidence, reports, generated artifacts, verification logs where appropriate.
Search/index	Fast finding/asset/evidence search if database queries become insufficient.
Cache	Short-lived derived data; never sole authority for trusted state.
Queue/event store	Asynchronous jobs, retries, workflow signals.

The exact technology selection is finalized in the implementation architecture and infrastructure specification.

16. Eventing & Asynchronous Processing

Long-running workflows SHALL be asynchronous.

Workflow	Pattern
Cloud discovery	Job → worker → progress/events → persisted results
Security scan	Job → rule engine → finding events/results
AI generation	Request → AI worker/service → structured result → candidate

Impact analysis	Candidate event →DAIL worker →impact result
Verification	Candidate →verification job →gate results
Promotion	Approved candidate →promotion worker/controller →trusted state
Reporting	Report job →artifact generation →evidence/report record

Events should be idempotently processed and carry correlation identifiers.

17. Cloud Integration Architecture

Provider adapters isolate external APIs from product logic.

Boundary	Responsibilities
Credential/Role Manager	Secure access references and authorization context.
Provider Adapter	API-specific calls and pagination/throttling handling.
Normalizer	Provider object →CloudShield asset.
State Reconciler	Match observations to existing assets.
Execution Adapter	Perform only explicitly authorized infrastructure actions.
Rate/Retry Controller	Bound retries and respect provider limits.

18. Security Architecture & Trust Boundaries

Primary trust boundaries:

1. User/browser to CloudShield API.
2. CloudShield services to transactional data stores.
3. CloudShield to cloud provider APIs.
4. CloudShield to LLM provider.
5. CloudShield to external integrations.
6. AI-generated content to deterministic product controls.
7. Candidate state to trusted state.

The highest-risk boundary is the transition from AI-generated/proposed state to infrastructure execution/trusted state. That boundary must be mediated by authorization, schema validation, DAIL, verification, and policy.

19. Identity, Authorization & Tenant Isolation

- Authentication is handled at the API boundary.
- Authorization must be enforced server-side on every protected resource.
- Organization →Project →Environment is the default isolation hierarchy.
- Cloud credentials and provider roles are scoped to intended environments.
- Internal services use service identities and least-privilege permissions.
- Privileged actions require stronger authorization and complete audit context.
- Tenant/project identifiers must be propagated through asynchronous jobs and events.
- Cross-tenant data access must be structurally prevented, not merely hidden in the UI.

20. Evidence, Audit & Observability Architecture

Telemetry type	Purpose
Structured logs	Debugging and operational diagnosis.
Metrics	Health, latency, throughput, failures, backlog.
Traces	Cross-service workflow visibility.
Evidence	Proof supporting findings, verification, and promotion.
Audit events	Governance record of actor/service actions.
Lifecycle history	Candidate/trusted-state transitions.
AI metadata	Model/provider/version/context references subject to policy.

A correlation ID should follow a workflow from API request through jobs, AI calls, DAIL lifecycle, verification, promotion, and evidence.

21. Infrastructure & Deployment Architecture

The product should be deployable on AWS for the initial implementation.

Conceptual topology:

Internet / Users → Secure Edge / Load Balancer → Web/API Runtime → Application + Workers → Database / Object Storage / Queue → AWS Integrations

- Private subnets should be used for stateful/internal services where appropriate.
- Secrets should be managed through a dedicated secret-management mechanism.
- Database access should not be directly exposed to clients.
- Worker execution should be isolated from public ingress where possible.
- Monitoring and audit pipelines should be available across all environments.

22. Reliability & Failure Architecture

Failure	Required architectural response
LLM unavailable	Fail safely; explanation/generation unavailable without corrupting product state.
Cloud API timeout	Bounded retry/backoff; preserve last known state and mark freshness.
Discovery partial failure	Persist successful results and explicit incomplete status.
Verification failure	Block promotion; retain failure evidence.
Promotion failure	Preserve last trusted-state reference; expose recovery path.
Queue duplicate	Idempotent job processing.
Database transient failure	Retry safely or return explicit failure; no false success.
Policy service unavailable	Fail closed for mandatory promotion decisions.

23. Scalability & Performance Architecture

- Stateless API/application components should scale horizontally.
- Discovery, security scanning, AI inference, impact analysis, and verification should scale independently as workers.
- Long-running work must not consume interactive API threads.
- Cloud provider calls require concurrency/rate controls.
- AI calls require timeout, retry, and concurrency budgets.
- Large evidence artifacts belong in object storage rather than transactional rows.
- Indexes and search structures should target common asset, finding, candidate, and evidence queries.
- Performance targets from the SRS become measured acceptance gates before production.

24. Configuration & Policy Architecture

Policies are a first-class control plane.

Policy domain	Examples
Access	Roles, permissions, environment access.
Security	Finding rules, severity mapping, suppressions.
Risk	Scoring weights and thresholds.
AI	Allowed models, data classes, tools, retention.
Remediation	Allowed change classes and required controls.
Promotion	Required verification and approval gates.
Environment	Different controls for development/staging/production.

Policy versions must be referenced by decisions so historical outcomes remain interpretable.

25. Environment Architecture

Environment	Architecture intent
Local	Developer feedback with synthetic/sandbox data.
Development	Integrated service and cloud-adapter testing.
Staging	Production-like end-to-end verification.
Production	Hardened security, monitoring, backup, approval, and recovery controls.

Configuration and infrastructure must be version controlled and environment-specific differences explicit.

26. Technology Selection Principles

- Prefer mature, well-supported components over unnecessary novelty.
- Use relational persistence for strongly consistent product state in the MVP.
- Use object storage for large immutable artifacts.
- Use a queue/job mechanism for asynchronous workflows.
- Use Terraform or equivalent infrastructure-as-code for reproducibility.
- Keep cloud-provider APIs behind adapters.
- Keep model-provider APIs behind an AI abstraction.
- Prefer a modular monolith initially unless measured scale or team boundaries justify service separation.

This specification intentionally defines architectural responsibilities before locking every library, framework, or managed service.

27. API / Service Contracts

Contract	Minimum contract contents
Asset API	Asset identity, provider identity, configuration references, lifecycle metadata.
Finding API	Finding ID, asset, rule version, severity, evidence, state.
Risk API	Risk value/category, inputs, scoring version, timestamp.
Dependency API	Source, target, relationship type, provenance, version/time.
AI API	Task, context references, model metadata, structured result, validation status.
Candidate API	Candidate/version, source, target, proposed changes, motivation, status.
DAIL API	Candidate, lifecycle state, impact, verification, promotion decision, evidence.
Evidence API	Artifact ID, type, source, subject, timestamp, integrity metadata.
Job API	Job ID, type, status, progress, retries, error, correlation ID.

Contracts should be schema-defined and versioned. Free-form prose is not an acceptable control interface for infrastructure-affecting operations.

28. Architectural Data Flows

28.1 Discovery flow

Cloud API → Provider Adapter → Normalizer → Identity Resolver → Asset Store → Configuration Snapshot → Event/Job Result

28.2 Security flow

Configuration → Rule Engine → Finding → Evidence → Risk Engine → Dashboard / AI Context

28.3 AI remediation flow

Finding + Authorized Context → AI Context Builder → Model Adapter → Structured Output → Validator → Remediation Candidate

28.4 Controlled change flow

Candidate →DAIL Identity →Dependency/Impact →Invariants →Verification →Policy/Approval →Promotion →Trusted State → Evidence

28.5 Monitoring flow

Cloud Observation →Reconciliation →Drift/Change Detection →Findings/Risk →Remediation Loop

29. Deployment Flow

- 8. Developer commits source and infrastructure definitions.
- 9. CI validates code, schemas, tests, security checks, and infrastructure changes.
- 10. Build artifacts are versioned.
- 11. Deployment progresses through development, staging, and production controls.
- 12. Post-deployment health checks verify service readiness.
- 13. Configuration and deployment versions are recorded.
- 14. Rollback uses known-good artifacts and infrastructure state where supported.

30. Architecture Decisions & Tradeoffs

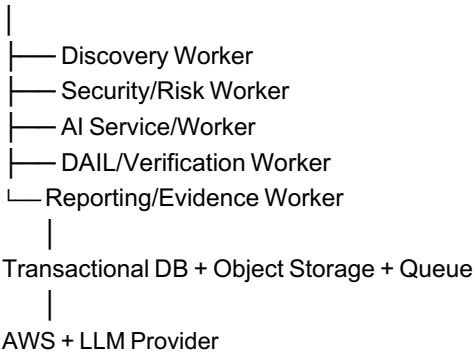
Decision	Rationale	Tradeoff
Modular monolith first	Reduces operational complexity for MVP.	Less independent scaling than full microservices.
Async workers	Cloud/AI/verification jobs are long-running.	Requires queue/job state management.
Relational system of record	Strong consistency for lifecycle/state.	Graph/search workloads may need specialized indexes later.
Provider adapters	Avoids cloud-specific coupling.	Adapter maintenance required.
AI abstraction	Enables hosted/local model substitution.	Adds interface/schema design work.
DAIL boundary	Makes deterministic lifecycle explicit.	Requires careful integration contracts.
Evidence as first-class data	Supports auditability and trust.	Storage/retention complexity.
Fail closed	Protects trusted state.	Can reduce automation during dependency outages.

31. MVP Architecture

The MVP should be implementable without a large distributed platform.

Recommended logical deployment:

Web UI + API/Application + Core Modules



- One primary cloud provider: AWS.
- One transactional database.
- One object-storage layer.
- One asynchronous job/queue mechanism.

- One AI abstraction with at least one model provider.
- DAIL integrated as a modular subsystem.
- Single deployment pipeline across environments.

32. Future Evolution

- Split high-load workers into independent services.
- Introduce a dedicated graph store if dependency scale warrants it.
- Add search infrastructure for large evidence/finding volumes.
- Support additional cloud providers through new adapters.
- Add local/open-weight model adapters and optional on-prem inference.
- Introduce advanced event streaming for continuous posture monitoring.
- Support multi-region/enterprise deployment.
- Expand policy engine and autonomous low-risk remediation under stronger controls.

33. Architecture-to-Requirements Traceability

Requirement domain	Architecture section(s)
Product boundaries	1–4
Cloud onboarding	8, 17, 19, 21
Asset/configuration	8, 15, 28
Security findings	9, 15, 20
Risk	9, 24, 28
Dependencies/impact	10, 13, 28
AI	11, 18, 24, 27, 28
Remediation	12, 13, 14, 27, 28
DAIL	13, 14
Evidence/audit	20, 15, 28
UX	6
APIs	7, 27
Security	18, 19, 21, 22
Reliability	22
Performance	23
Deployment	21, 25, 29
MVP	31

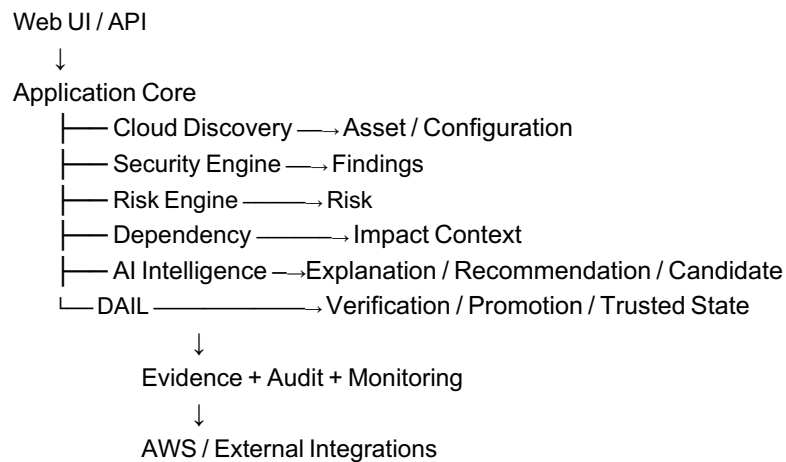
34. Architecture Acceptance Criteria

- The architecture defines CloudShield AI as the complete product and DAIL as a subsystem.
- Every major SRS capability has an architectural owner.
- AI provider and cloud provider dependencies are isolated behind explicit boundaries.
- Observed, proposed, verified, and trusted states are architecturally distinct.
- Infrastructure-affecting actions cannot bypass authorization, DAIL, verification, and policy controls.
- Long-running work is asynchronous and observable.
- Tenant/project/environment isolation is enforced server-side.
- Evidence and audit are integrated into important lifecycle paths.
- MVP can be implemented without premature microservice complexity.
- The architecture provides a credible path to local-model adoption later without requiring it for MVP.
- All architectural decisions can be traced to product requirements.

Appendix A — Canonical Architecture

Users / Automation





Master architectural rule: AI proposes; deterministic systems validate; governance decides; cloud APIs execute; evidence records; trusted state is earned.